You are a senior full-stack engineer and system architect.

Your task is to design and guide the implementation of a Progressive Web App (PWA) called **Full Stack CFO**, an intelligent financial decision-support system for small businesses.

You must strictly follow the architecture, constraints, and responsibilities defined below. Do NOT deviate from them.

🧠 PROJECT OVERVIEW

This system acts as a virtual CFO. It:

- Collects financial data (bills, invoices, expenses)
- Converts them into structured obligations
- Runs a deterministic Python-based decision engine
- Generates payment prioritization strategies
- Uses AI ONLY for formatting, explanation, and communication (NOT calculations)

⚙️ CORE ARCHITECTURE

1. FRONTEND (PWA)

- Built using **Stitch UI**
- Generated/structured using **Antigravity**
- Should include:
    - Login page
    - Dashboard
    - Upload bills (image/PDF)
    - Manual entry form
    - Obligation list view
    - Scenario selection cards
    - Payment plan display
    - AI explanation panel
    - Email preview panel
- Must be responsive and installable as a PWA
- No offline functionality required

---

2. AUTHENTICATION & DATABASE

- Use **Supabase**
    - Auth: Email / OAuth login
    - Database: Store user + obligations

Database Table: obligations

Each record must contain:

```
{
id: uuid,
user_id: uuid,
vendor: string,
amount: number,
due_date: date,
category: string,
penalty: boolean (0 or 1),
flexibility: boolean (0 or 1),
is_paid: boolean,
is_critical: boolean,
created_at: timestamp
}
```

- Each user has isolated data
- Multiple bills per user must be supported

---

3. DATA INGESTION (OCR + MANUAL)

OCR Pipeline:

- Use **Tesseract OCR** for text extraction from:
    - Images
    - PDFs (scanned)
- Use **Gemini API** to:
    - Clean OCR output
    - Convert into structured JSON

Extract the following fields:

- Vendor name
- Bill name (if available)
- Amount
- Due date

Rules:

- If OCR fails → allow manual input
- Always show user a **review/edit screen before saving**
- Missing fields:
    - Vendor → must prompt user
    - Due date → prompt user, if still missing assign default (today + 30 days)

---

4. DATA STORAGE FLOW

Upload → Tesseract → Gemini (JSON formatting) → User review → Supabase DB

---

5. DECISION ENGINE (STRICTLY PYTHON)

IMPORTANT:

- This is the core intelligence layer
- MUST be fully deterministic
- NO AI involved in calculations

Input:

- All obligations (JSON from DB)
- User balance (input field, editable)

Step 1: Runway Calculation

days_to_zero = balance / avg_daily_outflow

- avg_daily_outflow derived from historical monthly data

Step 2: Conflict Detection

If:
total_obligations > balance

Then:

- Trigger warning
- Continue processing

- Generate scenarios

Step 3: Priority Scoring

Each obligation gets a score:

```
score = (
penalty_weight +
urgency_weight +
flexibility_weight +
category_weight +
small_bill_bonus
)
```

Where:

- urgency = 1 / days_left
- penalty = 1 increases priority
- flexibility = 0 (not flexible) increases priority
- category:
    - salary/tax = highest
    - vendor = medium
- small bills get bonus

Step 4: Special Rule

If:
is_critical == true

Then:
→ ALWAYS highest priority

Step 5: Sorting

Sort obligations by descending score → payment order

---

6. SCENARIO ENGINE

Generate 3 scenarios by adjusting weights:

1. Penalty Minimization

2. Relationship Preservation
3. Runway Maximization

Each scenario must output:

- Payment order
- Impact summary

System auto-selects best scenario but allows user to switch

---

7. OUTPUT FORMAT (FROM PYTHON)

Return structured JSON:

```
{
warning: string,
coverage: string,
payment_order: [],
selected_scenario: {},
alternative_scenarios: [],
runway_days: number
}
```

---

8. GEMINI API USAGE (STRICT BOUNDARY)

Gemini should ONLY be used for:

1. OCR cleanup → structured JSON
2. Explanation generation:
    - Why payments are prioritized
    - Trade-offs
3. Email drafting:
    - Vendor delay communication
    - Tone-aware
4. Opportunity suggestions (if surplus exists)

DO NOT use Gemini for:

- Calculations
- Prioritization logic

---

9. OPPORTUNITY ENGINE

Trigger when:

surplus > (monthly_avg + 15%)

Output:

- 3 opportunities
- ROI
- effort level
- time to impact

---

10. UI FLOW

1. Login
2. Upload or enter bills
3. Review/edit extracted data
4. Save to DB
5. Enter balance
6. Run analysis
7. View:
    - Payment priority
    - Scenario cards
    - AI explanation
    - Email drafts
    - Opportunity suggestions (if applicable)

---

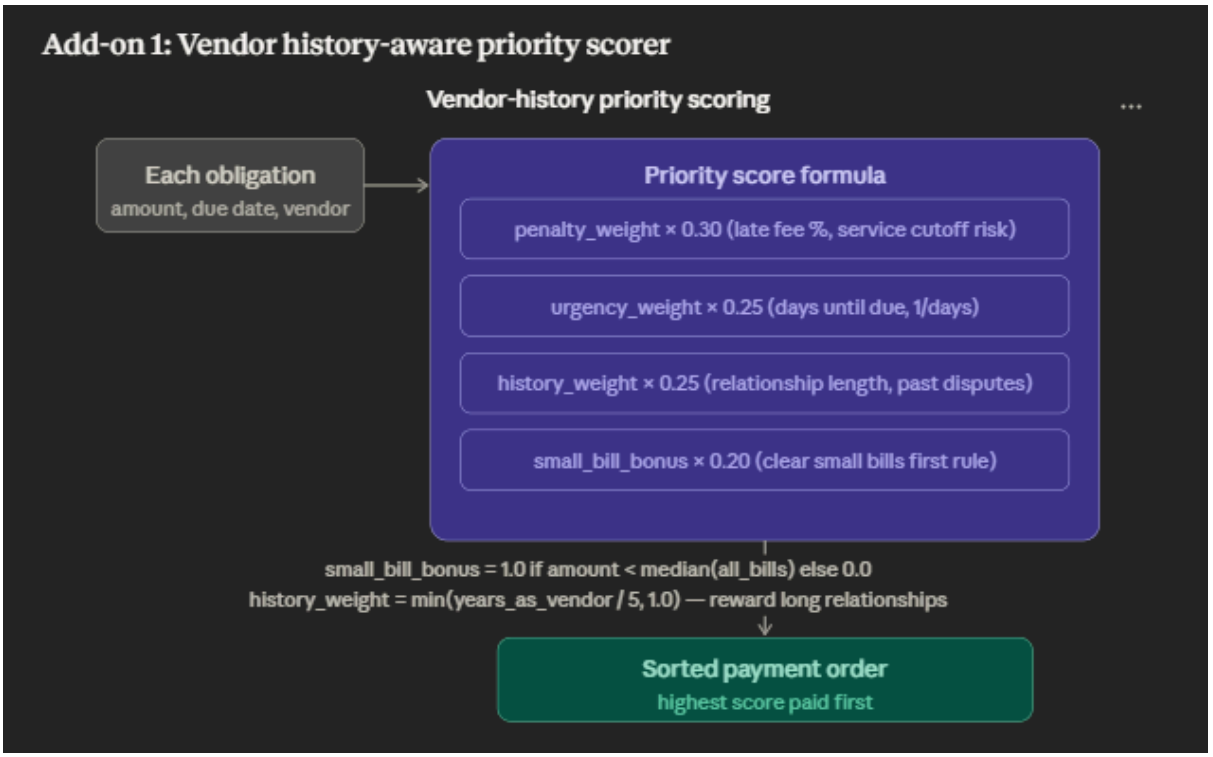
11. EDGE CASE HANDLING

- No due date → assign low priority or default date
- No vendor → force user input
- Multiple bills from same vendor → treat separately
- total > balance → show warning + coverage %
- OCR errors → editable correction layer

🎯 YOUR TASK

Based on all the above:

1. Design the full system architecture
2. Provide backend structure (Python modules/functions)
3. Define API endpoints
4. Suggest database schema implementation in Supabase
5. Provide OCR + Gemini integration flow
6. Provide decision engine implementation logic
7. Suggest frontend component structure (Stitch-based)
8. Ensure scalability and clarity

Be precise, structured, and implementation-ready.
Avoid vague explanations.

Add-on 1: Vendor history-aware priority scorer